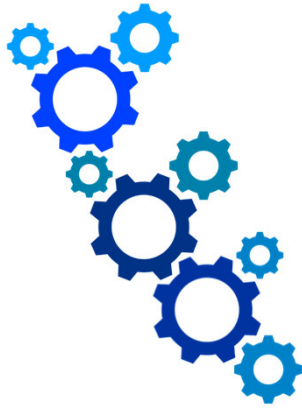


Technologies

Introductory concepts

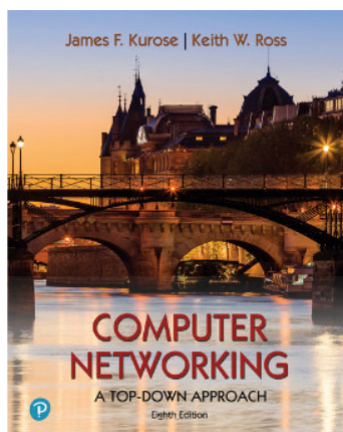


© M. C. Calzarossa 2025

Enterprise Digital Infrastructure

1

Resources: book



Chapter 2 Application Layer

Chapter 8 Security in Computer Networks

© M. C. Calzarossa 2025

Enterprise Digital Infrastructure

2

Resources: RFCs

RFC Editor

The Series

Document Retrieval
Errata
FAQ
xml2rfc FAQ
Format Change FAQ
History
About Us
Other Information

For Authors

Publication Process
Publication Queue
Style Guide

Mailing Lists

rfc-dist@rfc-editor.org is for RFC publication announcements. rfc-interest@rfc-editor.org is for discussion of the RFC series and related functions.

The RFC series

contains technical and organizational documents about the Internet, including the specifications and policy documents produced by four streams: the Internet Engineering Task Force (IETF), the Internet Research Task Force (IRTF), the Internet Architecture Board (IAB), and Independent Submissions.

Browse the RFC Index

HTML (ascending) • HTML (descending) • TXT • XML

Note: These files are large.

Browse RFCs by Status

Internet Standard

Draft Standard • Proposed Standard

Best Current Practice

Informational • Experimental • Historic

Uncategorized (Early RFCs)

• • • • •

Official Internet Protocol Standards

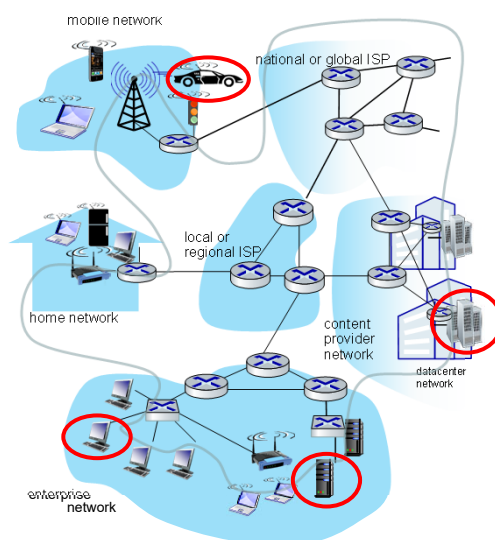
[RFC Status Changes](#)

© M. C. Calzarossa 2025

Enterprise Digital Infrastructure

3

Introduction



Networked applications

distributed applications written in any programming/scripting language (e.g., Java, C/C++/C#, Python, PHP, GO) consisting of *multiple components* that run on *heterogeneous* end systems and use the *network infrastructure* to communicate

© M. C. Calzarossa 2025

Enterprise Digital Infrastructure

4

Examples of networked apps

email
web
instant messaging
online games
video conferencing
audio/video streaming
VoD
VoIP
social networks
file sharing
remote login
and many many others ...

© M. C. Calzarossa 2025

Enterprise Digital Infrastructure

5

Architectural models

An architectural model refers to the *organization* of the various components of a networked application

The choice of the model is a major design choice

Two types of components:

- client components
- server components

Two main architectural models:

1. Client/server model

Hybrid model

2. Peer-to-peer (P2P) model

© M. C. Calzarossa 2025

Enterprise Digital Infrastructure

6

Client/server model

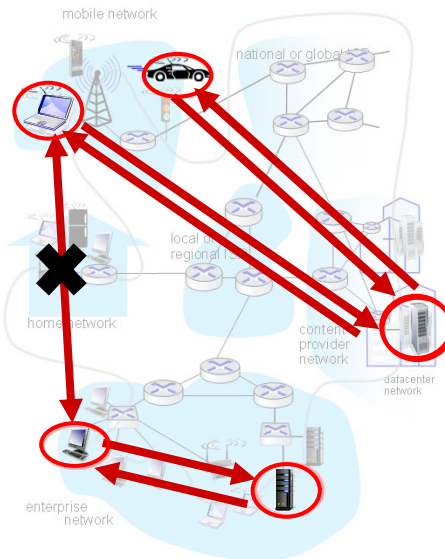
Centralized/hierarchical solution

The server component is the core of the application

The client components request the services provided by the server components

Communications between clients and servers

Examples: Web, email, DNS, ...



© M. C. Calzarossa 2025

Enterprise Digital Infrastructure

7

Role of server & client

Server component runs on a host computer (server)

- Always-on (*availability*)
- Static (*permanent*) IP address(es)
- Cloud infrastructure/data centers to ensure *scalability*

Client components run on user devices

- May be intermittently connected to the Internet
- May have dynamic IP addresses (e.g., assigned by ISPs using DHCP)
- Client components communicate with server component only
- Client components do not communicate directly with other client components

Infrastructure intensive model

© M. C. Calzarossa 2025

Enterprise Digital Infrastructure

8

Peer-to-peer model

Decentralized solution (no hierarchy)

Concept of “peers”, i.e., devices running applications/services

A peer requests services from other peers & provides services in return to other peers

Arbitrary end systems communicate directly

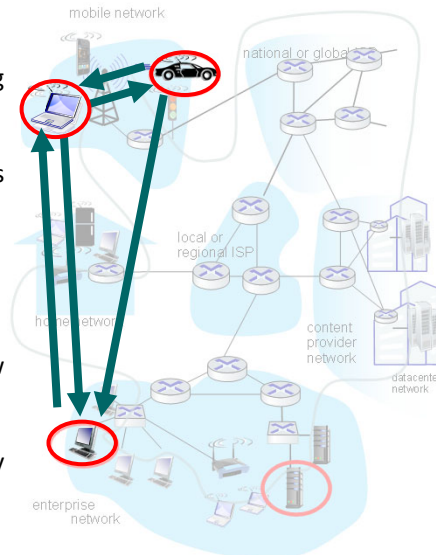
Self scalability (i.e., new peers bring new capacity and demands new services)

Peers are intermittently connected and may change their IP addresses

Examples: file sharing, blockchain, ...

© M. C. Calzarossa 2025

Enterprise Digital Infrastructure



9

C/S vs P2P

C/S

- Services owned/managed/operated by providers
- Expensive
- Infrastructure intensive model

P2P

- Services operated by the *community* for the *community*
- Cheap
- Concept of sharing (under the assumption of “good” behavior)
- Legal issues (e.g., copyrighted material)
- Security issues (e.g., trust of anonymous peers)
- Technical challenges (e.g., asymmetric bandwidth)
- Lack of incentives to share unused resources

© M. C. Calzarossa 2025

Enterprise Digital Infrastructure

10

Hybrid model

Mixed solution

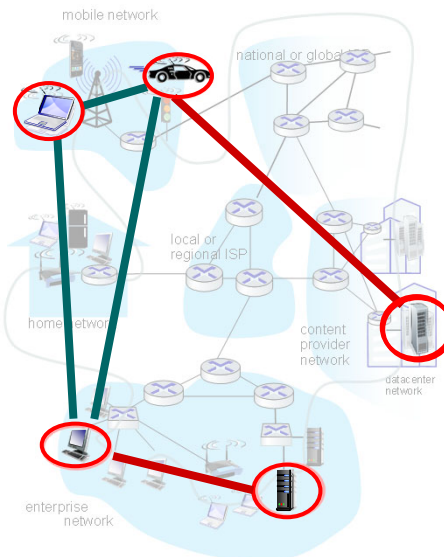
Services include a light “centralized” component (i.e., C/S) and “decentralized” components (i.e., P2P)

Control over the service with reduced cost

Examples:

Zoom/MS Teams: centralized servers in the cloud for authentication + P2P one-on-one conversations

Instant messaging: centralized server for client presence detection/location + client-client chats



© M. C. Calzarossa 2025

Enterprise Digital Infrastructure

11

Implementation of networked apps

A *process* is a program running on a host system

Pairs of processes running on the same host communicate using *inter-process communication* mechanisms (defined by operating systems)

Pairs of processes running on different (heterogeneous) hosts communicate by exchanging *messages*

Client process is the process that initiates the communication and requests the service

Server process is the process that waits to be contacted and eventually provides the requested service

P2P applications include both client and server processes

© M. C. Calzarossa 2025

Enterprise Digital Infrastructure

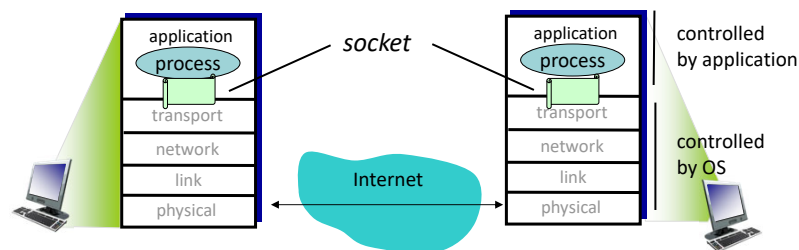
12

Sockets

A process sends/receives messages to/from its *socket* (corresponding to the exit/entry point of the process)

Sending process relies on transport infrastructure to deliver messages to the socket at the receiving process

Two sockets involved (one on each side)



© M. C. Calzarossa 2025

Enterprise Digital Infrastructure

13

Addressing

Each process must be uniquely identified for the message exchange to work

The IP address of the device on which the process is running is *necessary* for identifying a process, but it is not *sufficient*

Many processes might be running on the same device

The *identifier* of a process must include *both* the *IP address* of the device and the *port number* associated with the process

On the server side, some port numbers are *reserved* to specific services (e.g., port 80 for HTTP, port 443 for HTTPS, port 22 for SSH, port 53 for DNS)

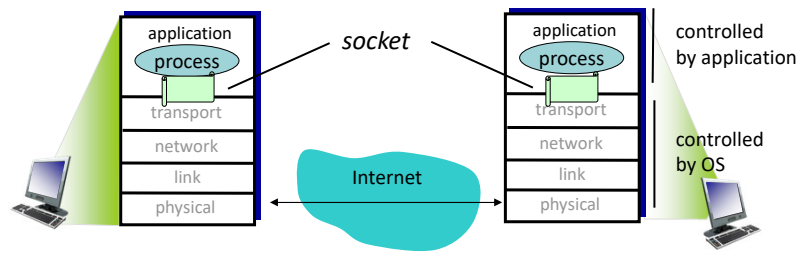
On the client side, each client process chooses a (random) port number where it will expect to receive responses from the server

© M. C. Calzarossa 2025

Enterprise Digital Infrastructure

14

Application vs transport protocols



© M. C. Calzarossa 2025

Enterprise Digital Infrastructure

15

Application requirements on transport service

Developers have to choose the most suitable transport protocol for their applications

The choice depends on *QoS requirements* of the applications with respect to *data transfer*

QoS requirements (*wish list*) are defined in terms of four dimensions:

- **Data integrity:** the reliability in the transfer between sending and receiving processes (i.e., tolerance to data losses)
- **Throughput:** the rate at which sending process can deliver data to receiving process (i.e., bandwidth requirements)
- **Time/delay:** the end-to-end delay between sending and receiving processes (i.e., timing requirements)
- **Security:** the privacy/confidentiality of data being transmitted (i.e., constraints on data in transit against security threats, such as eavesdropping, tampering and forgery)

© M. C. Calzarossa 2025

Enterprise Digital Infrastructure

16

Transport service requirements: examples

application	data loss	throughput	time sensitive
file transfer	no loss	elastic	no
e-mail	no loss	elastic	no
web browsing	no loss	elastic	no
real-time audio/video	loss-tolerant	audio: 5K-1Mbps video: 10K-5Mbps	yes, 10's msec
streaming	loss-tolerant	same as above	yes, few secs
interactive games	loss-tolerant	Kbps+	yes, 10's msec
text messaging	no loss	elastic	yes and no

© M. C. Calzarossa 2025

Enterprise Digital Infrastructure

17

Transport protocols

TCP

- + *reliable data transfer* between sending and receiving processes
- *no timing guarantee*
- *no minimum throughput guarantee*
- *no security* (TLS on top of TCP)
- *connection oriented*: setup between client/server processes (handshaking)

UDP

- *unreliable data transfer* between sending and receiving processes
- *no timing guarantee*
- *no minimum throughput guarantee*
- *no security* (DTLS on top of UDP)
- + *connectionless*: no setup between client/server processes

© M. C. Calzarossa 2025

Enterprise Digital Infrastructure

18

Application layer protocols

Protocols specify the *types* of messages exchanged between processes, their *syntax* and *semantics* and the *rules/actions* associated with messages (e.g., when and how processes send & respond to messages)

Two categories of protocols:

- *Open protocols* whose characteristics/behavior are fully specified within public documents (e.g., RFCs) to ensure interoperability
- *Proprietary protocols* whose specifications are not made publicly available to “protect” business (e.g., competitiveness)